

Tree Operations in C

Program:

```
#include<stdio.h>
#include<stdlib.h>
#include<stdbool.h>

struct BinaryST{
    struct BinaryST* lchild;
    int data;
    struct BinaryST* rchild;
};

struct BinaryST * tree = NULL;

struct BinaryST* getBinaryST(){
    struct BinaryST *bst = (struct
BinaryST*)malloc(sizeof(struct BinaryST));
    bst->lchild = NULL;
    bst->rchild = NULL;
}

void insertNode(struct BinaryST *root, int data){

    if(root == NULL){
        tree = getBinaryST();
        tree->data = data;
    }
    else{
        if(data > root->data){
            if(root->rchild == NULL){
                root->rchild = getBinaryST();
```

```

        root->rchild->data = data;
    }
    else{
        insertNode(root->rchild, data);
    }
}
else{
    if(root->lchild == NULL){
        root->lchild = getBinaryST();
        root->lchild->data = data;
    }
    else{
        insertNode(root->lchild, data);
    }
}
}
}

```

```

void inorder(struct BinaryST* root){
    if(root){
        inorder(root->lchild);
        printf("%d\t", root->data);
        inorder(root->rchild);
    }
}

```

```

void preorder(struct BinaryST* root){
    if(root){
        printf("%d\t", root->data);
        preorder(root->lchild);
        preorder(root->rchild);
    }
}

```

```

    }
}

void postorder(struct BinaryST* root){
    if(root){
        postorder(root->lchild);
        postorder(root->rchild);
        printf("%d\t",root->data);
    }
}

void search(struct BinaryST *root, int data){
    if(root == NULL){
        printf("Element doesn't exist..");
    }
    else{
        if(data == root->data){
            printf("%d found",root->data);
        }
        else{
            if(data > root->data){
                search(root->rchild, data);
            }
            else{
                search(root->lchild, data);
            }
        }
    }
}

void leafs(struct BinaryST *root){
    if(root != NULL){

```

```

        leafs(root->lchild);
        if(root->lchild == NULL && root->rchild == NULL){
            printf("%d\t",root->data);
        }
        leafs(root->rchild);
    }
}

```

```

int findDepth(struct BinaryST* root) {
    if (root == NULL)
        return 0;

    int leftDepth = findDepth(root->lchild);
    int rightDepth = findDepth(root->rchild);

    if (leftDepth > rightDepth)
        return leftDepth + 1;
    else
        return rightDepth + 1;
}

```

```

struct BinaryST* getSuccessor(struct BinaryST* node) {
    while (node && node->lchild != NULL)
        node = node->lchild;
    return node;
}

```

```

struct BinaryST* deleteNode(struct BinaryST* tree, int data) {
    if (tree == NULL)
        return NULL;

```

```

    // Step 1: Traverse the tree

```

```

if (data < tree->data)
    tree->lchild = deleteNode(tree->lchild, data);
else if (data > tree->data)
    tree->rchild = deleteNode(tree->rchild, data);
else {
    // Step 2: Node found

    // Case 1: No child
    if (tree->lchild == NULL && tree->rchild == NULL) {
        free(tree);
        return NULL;
    }

    // Case 2: One child, i.e. right child present
    else if (tree->lchild == NULL) {
        struct BinaryST* temp = tree->rchild;
        free(tree);
        return temp;
    }

    //case: One child, i.e. left child is present
    else if (tree->rchild == NULL) {
        struct BinaryST* temp = tree->lchild;
        free(tree);
        return temp;
    }

    // Case 3: Two children
    else {
        struct BinaryST* temp = getSuccessor(tree-
>rchild);
        tree->data = temp->data;
        tree->rchild = deleteNode(tree->rchild, temp-
>data);
    }
}

```

```

    }
}
return tree;
}

```

```

void parentwithChild(struct BinaryST* root){
    if(root != NULL){
        parentwithChild(root->lchild);
        if(root->lchild == NULL && root->rchild != NULL){
            printf("\n%d - %d",root->data, root->rchild-
>data);
        }
        else if(root->rchild == NULL && root->lchild != NULL){
            printf("\n%d - %d",root->data, root->lchild-
>data);
        }
        else if(root->rchild != NULL && root->lchild != NULL){
            printf("\n%d - %d, %d",root->data, root->lchild-
>data, root->rchild->data);
        }
        parentwithChild(root->rchild);
    }
}

```

// Queue node for holding BinaryST node pointers

```

struct QueueNode {
    struct BinaryST* treeNode;
    struct QueueNode* next;
};

```

// Queue structure

```

struct Queue {
    struct QueueNode* front;

```

```

    struct QueueNode* rear;
};

// Function to create an empty queue
struct Queue* createQueue() {
    struct Queue* q = (struct Queue*)malloc(sizeof(struct
Queue));
    q->front = q->rear = NULL;
    return q;
}

// Function to enqueue a tree node
void enqueue(struct Queue* q, struct BinaryST* node) {
    struct QueueNode* temp = (struct
QueueNode*)malloc(sizeof(struct QueueNode));
    temp->treeNode = node;
    temp->next = NULL;
    if (q->rear == NULL) {
        q->front = q->rear = temp;
        return;
    }
    q->rear->next = temp;
    q->rear = temp;
}

// Function to dequeue a tree node
struct BinaryST* dequeue(struct Queue* q) {
    if (q->front == NULL)
        return NULL;
    struct QueueNode* temp = q->front;
    struct BinaryST* node = temp->treeNode;
    q->front = q->front->next;
    if (q->front == NULL)

```

```

        q->rear = NULL;
    free(temp);
    return node;
}

bool isEmpty(struct Queue* q) {
    return q->front == NULL;
}

// Level Order Traversal
void levelTraversal(struct BinaryST* root) {
    if (root == NULL)
        return;

    struct Queue* q = createQueue();
    enqueue(q, root);

    while (!isEmpty(q)) {
        struct BinaryST* current = dequeue(q);
        printf("%d ", current->data);

        if (current->lchild != NULL)
            enqueue(q, current->lchild);
        if (current->rchild != NULL)
            enqueue(q, current->rchild);
    }

    free(q);
}

void main(){
    int choice;

```



```

int data;
printf("\n1 - Insert Node");
printf("\n2 - In-order Traversal");
printf("\n3 - Pre-order Traversal");
printf("\n4 - Post-order Traversal");
printf("\n5 - Search node");
printf("\n6 - Leaf nodes");
printf("\n7 - Depth of Tree");
printf("\n8 - Delete a node");
printf("\n9 - All parent with child nodes");
printf("\n10 - Lever-wise traversal");
printf("\n11 - Exit");

while(1){

    printf("\n\nEnter Choice: ");
    scanf("%d",&choice);
    switch(choice){
        case 1:
            printf("\nEnter the data: ");
            scanf("%d",&data);
            insertNode(tree,data);
            break;
        case 2:
            inorder(tree);
            break;
        case 3:
            preorder(tree);
            break;
        case 4:
            postorder(tree);
            break;
    }
}

```

```

    case 5:
        printf("\nEnter the data to search: ");
        scanf("%d",&data);
        search(tree, data);
        break;
    case 6:
        leafs(tree);
        break;
    case 7:
        data = findDepth(tree);
        printf("\nThe depth is: %d",data);
        break;
    case 8:
        printf("\nEnter the data to delete: ");
        scanf("%d",&data);
        deleteNode(tree,data);
        break;
    case 9:
        parentwithChild(tree);
        break;
    case 10:
        levelTraversal(tree);
        break;
    case 11:
        exit(0);
        break;
}

}

}

```

Output:

```
1 - Insert Node
2 - In-order Traversal
3 - Pre-order Traversal
4 - Post-order Traversal
5 - Search node
6 - Leaf nodes
7 - Depth of Tree
8 - Delete a node
9 - All parent with child nodes
10 - Lever-wise traversal
11 - Exit
```

Enter Choice: 1

Enter the data: 10

Enter Choice: 1

Enter the data: 20

Enter Choice: 1

Enter the data: 30

Enter Choice: 2

10 20 30

Enter Choice: 3

10 20 30

Enter Choice: 3

10 20 30

Enter Choice: 4

30 20 10

Enter Choice: 5

Enter the data to search: 20
20 found

Enter Choice: 6

30

Enter Choice: 7

The depth is: 3

Enter Choice: 8

Enter the data to delete: 30

Enter Choice: 9

10 - 20

Enter Choice: 10

10 20

Enter Choice: 11

